ME 523 – Computational Fluid Dynamics

Winter 2026

Final Project

Due date: Wednesday April 29, 2026 at 11:59 p.m. on Canvas.

Concepts: Solving the 2D low Mach number Navier–Stokes equations (with optional MPI parallelization).

Teamwork: Please turn in a single document per team of two–three, with your names on it. Provide a few sentences explaining in details the contribution of each team member to the overall project. All documents should be contained within a single PDF titled *team_name_ME523_final.pdf*.

Objectives: Write a code that solves the 2D low Mach number Navier–Stokes equations $+X$. Details on the solution methodology for the Navier–Stokes equations are provided in §1. The equations should be implemented using schemes with at least second order accuracy in both space and time. A two dimensional domain $[0, Lx] \times [0, Ly]$ should be discretized using at least 64 cells in x and 64 cells in y. For extra credit, work can be distributed over multiple processors to enable efficient computations, as outlined in §1.7. The extension $(+X)$ must either advance the modeling accuracy, computational performance, or physical representation of the flow. Example extensions are given in §2 of this document. Any extension chosen that is not provided in §2 must have been approved by Prof. Capecelatro prior to the assigned date of the project (Apr 20). Please answer the following questions:

(a) Show all of the equations being solved and clearly state any assumptions used.

(b) Summarize the implementation of the discretization employed (including discretization of the extension if applicable).

(c) Comment on the expected accuracy and stability of the method. In particular, what time step restriction do you expect to face? Does it match your observations?

(d) Vary the Reynolds number $\mathrm{Re} = 10$, $\mathrm{Re} = 1000$, and $\mathrm{Re} = \infty$ and comment on the observations. Is this expected?

(e) For at least one of the cases show a plot of the L2-norm of $\nabla \cdot \boldsymbol{u}$ at each time step. How is this value related to the convergence of the pressure Poisson equation?

(f) Provide a vector plot of the $\boldsymbol{u}(x, y)$ solution at three different times before steady-state is reached, and once steady-state is reached (approximately). Provide the times. Are these results expected? How could you ensure these results are correct?

(g) Provide a 1D plot of $u(x = Lx/2, y)$ at three different times before steady-state is reached, and once steady-state is reached. Provide the times. Compare to your expectations.

(h) Time your various subroutines. Which part of the solution process is the most expensive?

(i) In addition to the questions given above, you should also summarize the implementation of the Navier–Stokes extension $(+X)$ and provide an analysis of the flow dynamics. Details on the discretization of the extension must be included. For cases with scalar transport you should comment on the choice in discretization and its effect on robustness of the scheme (e.g., upwinding vs. QUICK). Example questions are given in §2.

(j) Please provide your code.

1 Incompressible Navier–Stokes solver

1.1 Incompressible Navier–Stokes equations

We are finally considering the two-dimensional Navier–Stokes equations in the incompressible limit and non-dimensional form:

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0, \tag{1}$$

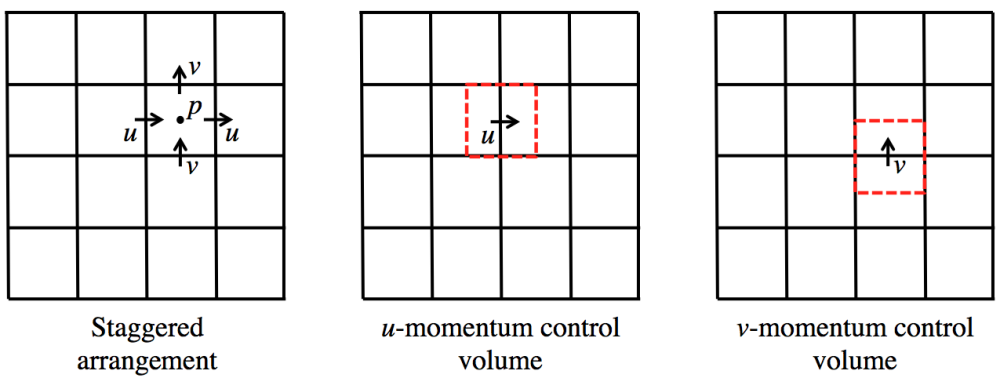
$$\frac{\partial u}{\partial t} + \frac{\partial uu}{\partial x} + \frac{\partial uv}{\partial y} = -\frac{\partial p}{\partial x} + \frac{1}{\mathrm{Re}}\left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}\right), \tag{2}$$

$$\frac{\partial v}{\partial t} + \frac{\partial uv}{\partial x} + \frac{\partial vv}{\partial y} = -\frac{\partial p}{\partial y} + \frac{1}{\mathrm{Re}}\left(\frac{\partial^2 v}{\partial x^2} + \frac{\partial^2 v}{\partial y^2}\right), \tag{3}$$

where Re is the Reynolds number of the flow field.

We want to solve for the velocity field $\boldsymbol{u} = (u, v)^\mathsf{T}$ in two dimensions.

1.2 Staggered variable arrangement



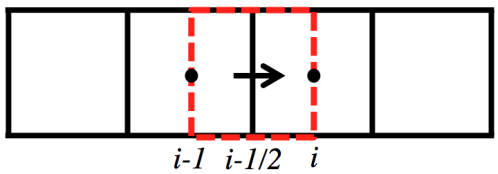
For convenience, we will store the data in a **staggered** layout: for cell (i, j), we store velocity u at the barycenter of the left face (i.e., $u_{i-\frac{1}{2},j}$), v at the barycenter of the bottom face (i.e., $v_{i,j-\frac{1}{2}}$), and the pressure p at the cell center (i.e., $p_{i,j}$). This is done to help us discretize the continuity equation (in our case, the solenoidal constraint on the velocity), since, to second order accuracy,

$$\nabla \cdot \boldsymbol{u}|_{i,j} \approx \frac{u_{i+\frac{1}{2},j} - u_{i-\frac{1}{2},j}}{\Delta x} + \frac{v_{i,j+\frac{1}{2}} - v_{i,j-\frac{1}{2}}}{\Delta y}. \tag{4}$$

With this approach, one has to be careful when discretizing the momentum equations. In particular, the control volume used for the u-momentum equation is different from the control volume used for the v-momentum equation, which is also different from the control volume used for writing the solenoidal condition. To clarify things, let us look at discretizing the various terms in the u-momentum equation for the control volume centered on $\left(i-\frac{1}{2}, j\right)$ with second order accuracy.

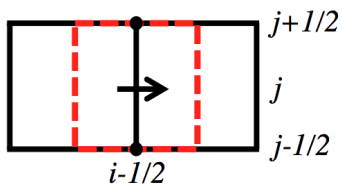
Convective term in x:

$$\left.\frac{\partial uu}{\partial x}\right|_{i-\frac{1}{2},j} \approx \frac{u_{i,j}^2 - u_{i-1,j}^2}{\Delta x} \tag{5}$$



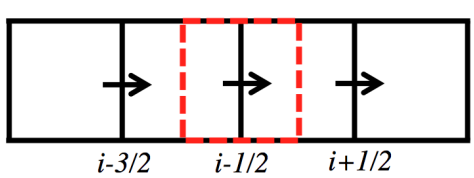
Convective term in y:

$$\left.\frac{\partial uv}{\partial y}\right|_{i-\frac{1}{2},j} \approx \frac{u_{i-\frac{1}{2},j+\frac{1}{2}} v_{i-\frac{1}{2},j+\frac{1}{2}} - u_{i-\frac{1}{2},j-\frac{1}{2}} v_{i-\frac{1}{2},j-\frac{1}{2}}}{\Delta y} \tag{6}$$



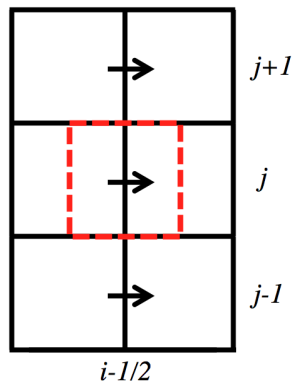
Viscous term in x:

$$\left.\frac{\partial^2 u}{\partial x^2}\right|_{i-\frac{1}{2},j} \approx \frac{u_{i+\frac{1}{2},j} - 2u_{i-\frac{1}{2},j} + u_{i-\frac{3}{2},j}}{\Delta x^2} \tag{7}$$



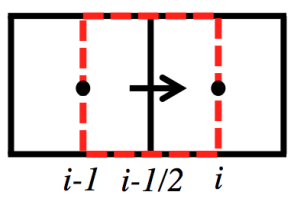
Viscous term in y:

$$\left.\frac{\partial^2 u}{\partial y^2}\right|_{i-\frac{1}{2},j} \approx \frac{u_{i-\frac{1}{2},j+1} - 2u_{i-\frac{1}{2},j} + u_{i-\frac{1}{2},j-1}}{\Delta y^2} \tag{8}$$



Pressure term in x:

$$\left.\frac{\partial p}{\partial x}\right|_{i-\frac{1}{2},j} \approx \frac{p_{i,j} - p_{i-1,j}}{\Delta x} \tag{9}$$



In addition, a few additional quantities are necessary to compute, such as

$$u_{i,j} = \frac{1}{2}\left(u_{i-\frac{1}{2},j} + u_{i+\frac{1}{2},j}\right), \tag{10}$$

$$u_{i-\frac{1}{2},j-\frac{1}{2}} = \frac{1}{2}\left(u_{i-\frac{1}{2},j} + u_{i-\frac{1}{2},j-1}\right), \tag{11}$$

and

$$v_{i-\frac{1}{2},j-\frac{1}{2}} = \frac{1}{2}\left(v_{i,j-\frac{1}{2}} + v_{i-1,j-\frac{1}{2}}\right). \tag{12}$$

Observations:

1. All these approximations are centered in space, and formally second order accurate.
2. When coding, you can simply use $u(i,j) = u_{i-\frac{1}{2},j}$, $v(i,j) = v_{i,j-\frac{1}{2}}$, and $p(i,j) = p_{i,j}$.
3. The v-momentum equation can be obtained readily from the proposed discretization of the u-momentum equation.

1.3 Temporal discretization – the fractional step method

We will follow a methodology described by Kim and Moin (1985), known as **fractional step method.** Their method combines an explicit Adams-Bashforth discretization of the convective term with a Crank-Nicholson discretization of the viscous term, and uses a projection idea to handle the pressure coupling.

Step 1. The momentum equation is advanced without a pressure term. We will use a delta notation to shorten the expressions for the spatial derivatives presented above. Similarly, the effect of staggering is ignored to simplify the expressions. Finally, H is used as shorthand for the convective term.

$$\frac{\boldsymbol{u}^{\star}_{i,j} - \boldsymbol{u}^{n}_{i,j}}{\Delta t} = \frac{3}{2} H^{n}_{i,j} - \frac{1}{2} H^{n-1}_{i,j} + \frac{1}{2\,\mathrm{Re}} \left(\frac{\delta^2}{\delta x^2} + \frac{\delta^2}{\delta y^2} \right) \left(\boldsymbol{u}^{\star}_{i,j} + \boldsymbol{u}^{n}_{i,j} \right). \tag{13}$$

$H^n_{i,j}$ represents the convective term written from the velocity at time n, while $H^{n-1}_{i,j}$ represents the convective term written from the velocity at time $n-1$. Adams-Bashforth consists of a linear extrapolation of the convective term from $n-1$ and n to $n+\frac{1}{2}$, providing second order accuracy.

The viscous term is based on Crank-Nicolson, which is implicit and second order accurate. Combining this equation with an approximate factorization idea, we can rewrite it as a two-step update that involves only tridiagonal solves:

$$\left(1 - \frac{\Delta t}{2} \frac{1}{\mathrm{Re}} \frac{\delta^2}{\delta x^2}\right) \Delta \boldsymbol{u}^{\star\star}_{i,j} = \frac{\Delta t}{2} \left(3 H^{n}_{i,j} - H^{n-1}_{i,j}\right) + \frac{\Delta t}{\mathrm{Re}} \left(\frac{\delta^2}{\delta x^2} + \frac{\delta^2}{\delta y^2} \right) \boldsymbol{u}^{n}_{i,j}, \tag{14}$$

$$\left(1 - \frac{\Delta t}{2} \frac{1}{\mathrm{Re}} \frac{\delta^2}{\delta y^2}\right) \Delta \boldsymbol{u}^{\star}_{i,j} = \Delta \boldsymbol{u}^{\star\star}_{i,j}, \tag{15}$$

followed by the velocity update $\boldsymbol{u}^{\star}_{i,j} = \boldsymbol{u}^{n}_{i,j} + \Delta \boldsymbol{u}^{\star}_{i,j}$. Note that tridiagonal Gauss elimination is ideally suited for equations 14 and 15.

Step 2. A geometric projection is used to obtain an equation for the pressure. We want to add a pressure term to take us from $\boldsymbol{u}^{\star}_{i,j}$ to $\boldsymbol{u}^{n+1}_{i,j}$, such that $\nabla \cdot \boldsymbol{u}^{n+1}\big|_{i,j} = 0$. If we start by writing that

$$\frac{\boldsymbol{u}^{n+1}_{i,j} - \boldsymbol{u}^{\star}_{i,j}}{\Delta t} = -\nabla p^{n+1}\big|_{i,j}, \tag{16}$$

then $\nabla \cdot \boldsymbol{u}^{n+1}\big|_{i,j} = 0$ implies that

$$\nabla \cdot \left(\nabla p^{n+1}\right)_{i,j} = \frac{1}{\Delta t} \nabla \cdot \boldsymbol{u}^{\star}\big|_{i,j}. \tag{17}$$

Crucial observation: The Laplacian term in the pressure Poisson equation (the $\nabla \cdot \nabla$ term on the left-hand side of equation 17) **must** be discretized exactly using the gradient and divergence operators in steps 2 and 3, i.e., equation 4 and equation 9.

Step 3. The velocity must be corrected:

$$\boldsymbol{u}_{i,j}^{n+1} = \boldsymbol{u}_{i,j}^{\star} - \Delta t \, \nabla p^{n+1}\big|_{i,j} \tag{18}$$

1.4 Initial conditions

Depending on the flow you choose to solve, a precise initial condition may or may not be important. In many cases, choosing $\boldsymbol{u}\,(\boldsymbol{x},\, t{=}0) = 0$ with appropriate boundary conditions is sufficient.

1.5 Boundary conditions

Boundary conditions must be imposed at $x = 0$, $x = Lx$, $y = 0$, and $y = Ly$ surfaces for both the velocity and pressure. Depending on the flow under consideration, Dirichlet, Neumann, or periodic boundary conditions should be applied to the velocity field. The best boundary condition for the pressure (for non-periodic boundaries) is the application of zero normal gradients.

Symmetric boundary conditions

In many flows, it is reasonable to consider symmetric boundary conditions for u, v, and p. Applying symmetric boundary conditions at the top and bottom of the domain (i.e., the $y = 0$ and $y = Ly$ bounds) translates into:

1. Neumann conditions for the pressure, i.e.,

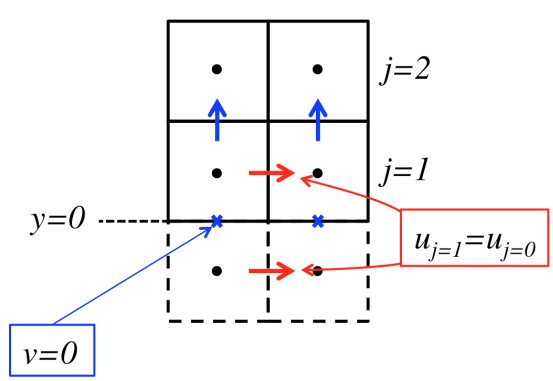
$$\frac{\partial p}{\partial y}\,(x, y = 0, t) = \frac{\partial p}{\partial y}\,(x, y = Ly, t) = 0. \tag{19}$$

2. Neumann conditions for the streamwise velocity u, i.e.,

$$\frac{\partial u}{\partial y}\,(x, y = 0, t) = \frac{\partial u}{\partial y}\,(x, y = Ly, t) = 0. \tag{20}$$

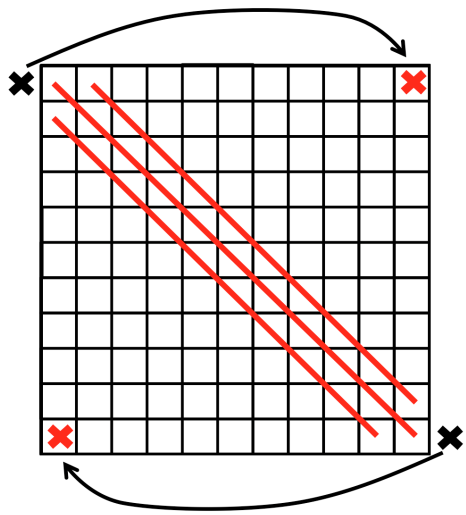
3. Dirichlet condition for the vertical velocity v, namely,

$$v\,(x, y = 0, t) = v\,(x, y = Ly, t) = 0. \tag{21}$$



A brief note on periodic boundary conditions

I asked you to use Crank-Nicolson, an implicit scheme, for handling the temporal integration of the viscous terms. Combined with an ADI approach, such a scheme will lead to two successive tri-diagonal matrix inversions. In class, we covered in details how to implement efficiently a Gauss inversion for tri-diagonal matrices. However, as shown in the sketch below, when solving the linear system with periodic boundary conditions in the y direction, you should obtain two additional entries in your operator, making it no longer tri-diagonal...



While this is not really an issue, it does prevent you from using the tri-diagonal Gauss solver discussed in class as is. You might have decided to use another solver anyway, which is fine. Tri-diagonal *periodic* Gauss solvers exist and are not particularly complicated, but do require a bit more work to be implemented. To avoid adding any complexity, we can take an even simpler approach. Assuming the flow is laminar and symmetric, it is reasonable to change from periodic boundary conditions to symmetric without a large consequence on the results, and will avoid changing the nature of the tri-diagonal operator.

1.6 On mass conservation and a well-defined Pressure Poisson equation

One last tricky consideration needs to be discussed: the pressure Poisson equation (equation 17) only admits a solution if the volume integral of its right-hand side **is zero**. This is not guarantied in the framework described heretofore. Hence, it needs to be enforced manually, and at each time step. Obviously, from Gauss' theorem, the integral of the divergence of $\boldsymbol{u}^\star$ is equal to the net volumetric flow through our computational domain, i.e.,

$$\int_{x=0}^{x=Lx}\int_{y=0}^{y=Ly} \nabla\cdot\boldsymbol{u}^\star dxdy = \int_{y=0}^{y=Ly} u^\star\left(x=0,y\right)dy - \int_{y=0}^{y=Ly} u^\star\left(x=Lx,y\right)dy. \tag{22}$$

If the left-hand side of this equation has to be zero, then that means that our exit ($x = Lx$) mass flow rate has to be constrained to match our inlet mass flow rate, i.e.,

$$\int_{y=0}^{y=Ly} u^\star\left(x=Lx,y\right)dy = \int_{y=0}^{y=Ly} u^\star\left(x=0,y\right)dy. \tag{23}$$

If the left boundary is fixed by a Dirichlet condition (i.e., an inflow condition), then we know the mass flow rate coming in the domain. In this case, the $u^\star$ velocity at $x = Lx$ (obtained at the end of step 1) needs to be corrected (through an additive or multiplicative correction) to ensure the same mass flow rate exits the computational domain. This should not be a surprise, since the flow is incompressible! To test your implementation, you should output the maximum value of the divergence field Eq. (4) at each time step.

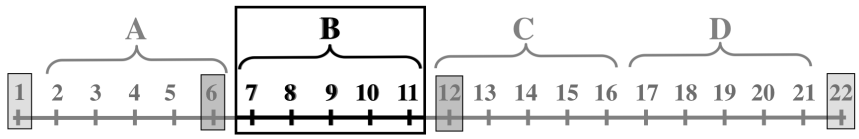
1.7 Parallelization (extra credit)

To enable efficient computations, the code should be parallelized using MPI (refer to HW1 for an example using textscPython). In HW1, a pencil (1D) decomposition was considered. Here, you may consider extending this to two dimensions with NP_x processors in x and NP_y processors in y, resulting in a total of $NP_x \times NP_y$ processors. Each processor will therefore own a sub domain with nx/NP_x cells in x and ny/NP_y cells in y. As shown below, if the global grid indices range from imin to imax in x, each processor will own the physical grid from imin_ to imax_. Because the governing equations involve derivatives, information from imin_-1 and imax_+1 are needed. Therefore, each processor owns a local extent of the grid that ranges from imino_ to imaxo_ (which includes overlapping points with the neighboring boundaries). These same ideas apply to the y direction as well.

Special care needs to be taken at processor boundaries when computing derivatives. This will require MPI_send and MPI_recv calls to neighboring processors to gain access to boundary data. Execution time per timestep should be reported using MPI_Wtime. Note that while parallelization of the linear solvers can be done, it is beyond the scope of this project. Therefore, the Parallel High Performance Preconditioners library (HYPRE) should be used to solve the pressure Poisson equation and implicit treatment of the viscous terms. The latest version of HYPRE can be downloaded at `http://computation.llnl.gov/projects/hypre-scalable-linear-solvers-multigrid-methods/software`.

Please report on the following aspects:

(c) How does the total time per timestep change with total number of processors? Is this expected? Why or why not?



- For processor B:
  - Global indices
    - Excluding overlap:
      - **imin = 2**
      - **imax = 21**
    - Including overlap
      - **imino = 1**
      - **imaxo = 22**
  - Local indices
    - Excluding overlap:
      - **imin_ = 7**
      - **imax_ = 11**
    - Including overlap
      - **imino_ = 6**
      - **imaxo_ = 12**

(d) How much does timing vary between processors? Is this expected? Why or why not?

(e) For a given number of processors, how is timing affected when changing the number of processors per direction? Is this expected? Why or why not?

1.8 Grading strategy

The final project will be graded out of 45 points. Below are details on how points will be assigned.

- *Flow solver (**25 pts**)*
  - Show equations and list assumptions (+4)
  - Show discretization (+4)
  - Lists expectations for accuracy/stability (+3)
  - Performs Reynolds number analysis (+3)
  - Reports 1D and 2D plots with discussion (+3)
  - Timing analysis (+3)
  - Working code (+5)
- *Extension +X (**20 pts**)*
  - Clearly shows equations and lists assumptions (+4)
  - Shows discretization (+4)
  - 1D/2D plots of results with discussion (+4)
  - Working code (+8)
- *Parallelization (extra credit; **10 pts**)*
  - Working code with MPI (+2)
  - Overview of MPI functions used (+2)
  - Timing analysis (+2)
  - Demonstration of parallel performance (i.e., speed-up and scale-up) (+4)

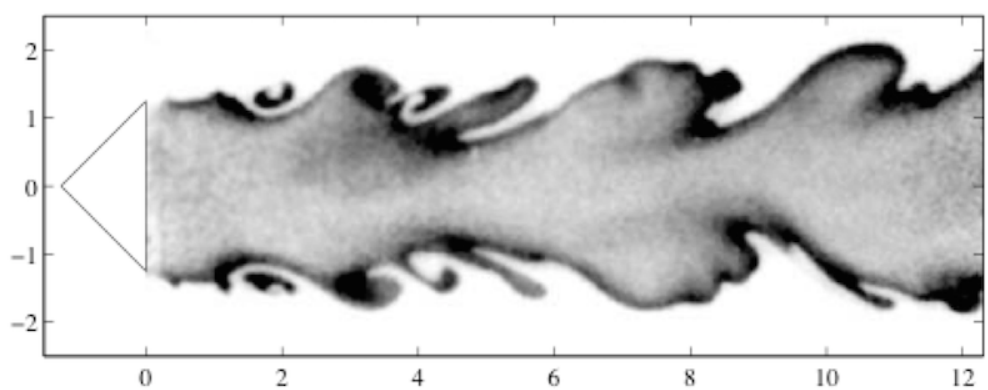
2 Extension ($+X$) to the Navier–Stokes equations

In addition to the implementation of the incompressible Navier–Stokes equations as described above, you may choose **one of the example extensions below, or choose an extension of your own**.

2.1 A simple premixed combustion model

In this example you will explore a model reactive flow in an augmentor (i.e., an aircraft engine afterburner configuration). In such a configuration, a mixture of fuel and oxidizer flows in the turbine exit channel, and our goal is to burn it completely in order to convert chemical energy into kinetic energy of the exhausted gases, ultimately leading to a significant augmentation of engine thrust.

While the gas mixture can be very easily ignited, the flow speed in the channel is much higher than the flame speed. Consequently, once ignited, the flame will be blown off the end of the channel if a carefully designed **flame holder** is not used. A flame holder is simply an obstacle placed in the flow in order to generate a low-speed region in which the flame can stabilize. An example of such a flow, taken from experimental visualizations by Knikker, is shown below.



We will handle combustion using two scalars, ϕ to represent the amount of combustible material available, and T to represent temperature. These scalars will obey the equations

$$\frac{\partial \phi}{\partial t} + \nabla \cdot (\boldsymbol{u}\phi) = \mathcal{D}\nabla^2 \phi + \dot{\omega}_\phi, \tag{24}$$

and

$$\frac{\partial T}{\partial t} + \nabla \cdot (\boldsymbol{u}T) = \mathcal{D}\nabla^2 T + \dot{\omega}_T. \tag{25}$$

In these, the combustion source terms are taken to be

$$\dot{\omega}_\phi = -\dot{\omega}, \tag{26}$$

and

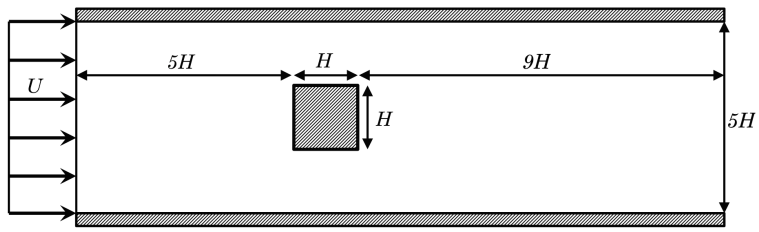
$$\dot{\omega}_T = A\,\dot{\omega}, \tag{27}$$

where we consider the following Arrhenius reaction

$$\dot{\omega} = \phi \left(\frac{T}{T_b}\right)^3 e^{-T_a/T}. \tag{28}$$

In these expressions, $\mathcal{D} = 10^{-4}$, $A = 1000$, $T_a = 1500$, and $T_b = 1000$. It is convenient to store the scalar at the cell center, like the pressure. Solve this equation using second order accuracy in time (AB for convection, CN for diffusion using ADI), second order central differencing in space for the diffusion term, and implement first order unwinding, second order central differencing, and third order QUICK for the spatial discretization of the convective term.

A flow configuration corresponding to a wake behind a square obstacle will be considered. I suggest that we consider a domain of $15H \times 5H$ as shown below



The following boundary conditions should be applied

- no-slip conditions along all four sides of the square obstacle,
- no-slip conditions for the top and bottom wall,
- Dirichlet boundary for the velocity at the inflow (where $\boldsymbol{u} = (U, 0)^\mathsf{T}$),
- Dirichlet conditions for the scalars at the inflow with $\phi(y,t) = 1$ and $T(y,t) = 300$,
- Neumann condition on the velocity and scalars at the outflow,
- and Neumann conditions on pressure for all surfaces.

We will vary the Reynolds number, define as $\mathrm{Re} = UH/\nu$ from 10 to 1000, by adjusting the kinematic viscosity ν, with $U = 10$ and $H = 1$. For sufficient resolution you should choose a mesh spacing $\Delta x = \Delta y = H/20$, i.e., a 300×100 mesh. For initial conditions, use $\boldsymbol{u} = 0$, $\phi = 1$, and $T = 300$ except in a $H \times H$ region behind the square where $T = 2500$ (this will serve to ignite the flame). Please run until $tU/H = 100$ to assess the capability of that configuration of holding the flame in place.

Note on the outflow: as described in §1.6, you will need to ensure that the RHS of the pressure Poisson equation is exactly zero. To that end, you should adjust the outflow velocity so that its bulk value is equal to U (implying that mass is exactly conserved).

Please report on the following aspects:

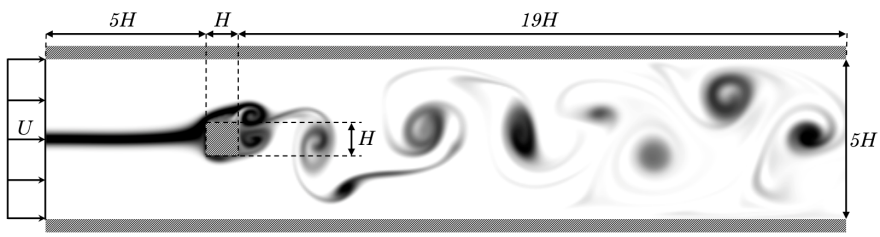
(a) Provide a 2D plot of the velocity field at $tU/H = 100$.

(b) Provide 2D plots of T (with a colormap between 300 and 2500) at $tU/H = 25$, 50, 75, 100. Is the flame holder working?

(c) How does the flow compare at different Reynolds numbers? How do the different discretization schemes affect the results at different Reynolds numbers?

2.2 Subgrid-scale turbulence model

Consider the flow configuration described in §2.1 (but without reaction terms or temperature) on a domain $25H \times 5H$ with $\Delta x = \Delta y = H/20$ as shown below. We are going to increase the Reynolds



number quite a bit (to 10,000). As a result, we will need a turbulence model of some sort for our simulation to make sense. I propose that we can build a simple large-eddy simulation (LES) model by making the following assumptions:

- The simulation is 2D while turbulence is inherently 3D, so we have to assume that the 3D nature of the turbulent eddies in this flow is unimportant. This is a poor assumption, but cost constraints prevent us from going 3D.
- We will consider that the use of an under-resolved mesh leads to implicit filtering of our variables (i.e., the fluctuations on the sub-grid level are not resolved, hence they can be considered to be filtered out). As a result, we will not worry about filtering variables in our LES – our computed solution will be considered to be automatically (implicitly) filtered. This is a standard approach in LES modeling.
- We will use a constant coefficient Smagorinsky model, with a constant $C_S = 0.15$. This is a fairly standard (somewhat outdated) approach in LES modeling.
- We will take some liberty with the form of the closure model in order to make its implementation as simple as possible. This is wrong, but not excessively problematic.
- We will assume that turbulent sub-filter transport of our scalars can be modeled similarly as turbulent sub-filter transport of momentum. This is a naive but acceptable assumption.

In the context of LES, our momentum equation is now going to be

$$\frac{\partial \boldsymbol{u}}{\partial t} + \nabla \cdot (\boldsymbol{u} \otimes \boldsymbol{u}) = -\nabla p + \nu \nabla^2 \boldsymbol{u} - \nabla \cdot \boldsymbol{\tau}^r, \tag{29}$$

where $\boldsymbol{u}$ is now the filtered velocity, and $\boldsymbol{\tau}^r$ is the anisotropic part of the residual tensor due to sub-filter velocity fluctuations (I am assuming that $\rho = 1$). Here, the isotropic part of the residual stress tensor is already included in a modified pressure called p. We need to model the last term in order to close this equation. Following an eddy viscosity assumption, we write

$$\boldsymbol{\tau}^r \approx -\nu_t \left(\nabla \boldsymbol{u} + \nabla \boldsymbol{u}^{\mathsf{T}} \right), \tag{30}$$

where ν_t is the eddy viscosity that we model following Smagorinsky as

$$\nu_t = (C_S \Delta)^2 \, S. \tag{31}$$

C_S is the Smagorinsky constant, which we will take as equal to 0.15. Δ is the filter size, which for us can simply be $\Delta = \Delta x = \Delta y$. Finally, S is the norm of the filtered rate of strain tensor (i.e.,

the rate of strain tensor computed based on the filtered velocity $\boldsymbol{u}$, i.e., the velocity in your code), defined as

$$S = \sqrt{2\boldsymbol{S}:\boldsymbol{S}}, \tag{32}$$

where

$$\boldsymbol{S} = \frac{1}{2}\left(\nabla\boldsymbol{u} + \nabla\boldsymbol{u}^{\mathsf{T}}\right). \tag{33}$$

That completes the definition of the model – everything can be computed from known variables at this point. However, the RHS of Eq. (2) is somewhat more complex to implement than the viscous term you have already in your code. Therefore, I propose that we simplify this term by writing the incorrect but not outlandish approximation

$$\nabla\cdot\left(\nu_t\left(\nabla\boldsymbol{u} + \nabla\boldsymbol{u}^{\mathsf{T}}\right)\right) \approx \nu_t\nabla^2\boldsymbol{u}. \tag{34}$$

With this, our new momentum equation is

$$\frac{\partial\boldsymbol{u}}{\partial t} + \nabla\cdot(\boldsymbol{u}\otimes\boldsymbol{u}) = -\nabla p + (\nu+\nu_t)\nabla^2\boldsymbol{u}, \tag{35}$$

where the effective viscosity $(\nu+\nu_t)$ will have to be computed everywhere in the domain at each time step according to Eq. (3). The initial/boundary conditions for velocity and pressure described in §2.1 should be employed.

Note that the same exercise is needed for the scalar transport equation! In this case you can think of the scalar as a very low concentration of dye. For simplicity, I propose that we follow the exact same procedure that was done for momentum, resulting in a general scalar equation

$$\frac{\partial\phi}{\partial t} + \nabla\cdot(\boldsymbol{u}\phi) = (\mathcal{D}+\mathcal{D}_t)\nabla^2\phi. \tag{36}$$

Invoking an assumption of unity turbulent Schmidt number, we can use $\mathcal{D}+\mathcal{D}_t = \nu+\nu_t$. Solve this equation using second order accuracy in time (AB for convection, CN for diffusion using ADI), second order central differencing in space for the diffusion term, and implement third order QUICK for the spatial discretization of the convective term. For boundary conditions, use Neumann at the top, bottom, and right boundaries. At the left boundary please use a Dirichlet condition with the profile

$$\phi(y,t) = \exp\left(-\left(\frac{y}{0.2H}\right)^2\right), \tag{37}$$

assuming that the domain is centered at $y=0$.

In summary: At each time step, you will need to compute S following Eqs. 32 and 33 everywhere in your domain (do not worry about boundary conditions, it is just a model so accuracy is not critical). Then, you will compute the viscosity field according to Eq. 31, and use it to advance the velocity according to Eq. 36. You will use the viscosity field as your diffusivity field for all your scalars as well.

Please report on the following aspects:

(a) Provide a 2D plot of the velocity field at $tU/H = 100$ for $\mathrm{Re} = 10,000$.

(b) Provide 2D plots of the normalized effective viscosity $(\nu+\nu_t)/\nu$ at $tU/H = 25$, 50, 75, 100. Comment on the location of maximum effective viscosity. Is this expected?

(c) How does the flow compare at different Reynolds numbers? How high of a Reynolds number can you achieve?

2.3 Particle-laden jet flow

In this example, you will solve the 2D Eulerian-Lagrangian equations describing the flow of particles suspended in a jet. In this approach, the incompressible Navier–Stokes equations are solved in an Eulerian frame of reference (as described in §1) and the particles are tracked individually in a Lagrangian manner. We will make a point-particle assumption, whereby the particles are assumed to be much smaller than the smallest scale of the fluid, and thus the particle size is negligible. In addition, we will assume one-way coupling between the phases, such that the fluid affects the particles, but the particles have no effect back on the fluid. Interparticle collisions will also be ignored. While these may appear to be very strong modeling assumptions, these simplifications are commonly used in particle-laden flow research.

Fluid phase description: We are proposing to compute a 2D planar jet flow on a domain $[0,2]\times[0,1]$ composed of 128 cells in x and 64 cells in y. To do this, we will provide a jet-like Dirichlet condition on the left of the domain, namely

$$u\left(x=0,y\right) = \exp\left(-\frac{\left(y-0.5\right)^2}{R^2}\right), \tag{38}$$

$$v\left(x=0,y\right) = 0, \tag{39}$$

where $R=0.05$ is a model for the jet radius. A compatible, and classically used boundary condition for the pressure is a Neumann condition, i.e., $\frac{\partial p}{\partial x}\left(x=0,y\right)=0$. For the top and bottom of the domain (i.e., the $y=0$ and $y=1$ bounds), we will use symmetric conditions for u, v, and p. For the right of the domain, i.e., $x=2$, we will use a Neumann boundary condition for u, v, and p to represent the outflow. The flow should be initialized with $\boldsymbol{u}=0$.

Particle phase description: The particle trajectories can be computed using Newton's second law as

$$\frac{\partial \boldsymbol{x}_p}{\partial t}=\boldsymbol{u}_p \quad \text{and} \quad \frac{\partial \boldsymbol{u}_p}{\partial t}=\frac{1}{St}\left(\boldsymbol{u}-\boldsymbol{u}_p\right), \tag{40}$$

where $\boldsymbol{x}_p=(x_p,y_p)$ is the particle position, $\boldsymbol{u}_p=(u_p,v_p)$ is the particle velocity, and St is the Stokes number that represents the ratio of particle to fluid timescales. The right-hand side of the particle velocity equation is the non-dimensional Stokes drag, and requires interpolating the fluid velocity $\boldsymbol{u}(\boldsymbol{x},t)$ from the grid to the particle location. For this project, linear interpolation of the fluid velocity to the particle location is acceptable. This requires determining the (i,j) grid cell each particle belongs to. Because we are considering a uniform mesh, this can easily be computed using the particle location and grid spacing, i.e., $i=\text{ceil}(x_p/\Delta x)$ and $j=\text{ceil}(y_p/\Delta y)$, where $\text{ceil}(x)$ is the ceiling operator that rounds each element of x to the nearest integer greater than or equal to that element.

Particles will be injected at the left boundary ($x_p=0$) with a random position between $0.45<y_p<0.55$ with the same velocity profile given for the fluid phase at the inflow, i.e.,

$$u_p\left(t\right) = \exp\left(-\frac{\left(y_p-0.5\right)^2}{R^2}\right), \tag{41}$$

$$v_p\left(t\right) = 0, \tag{42}$$

Particles that leave the domain (i.e., $x_p>2$, $y_p<0$, or $y_p>1$) should be removed from memory. The mass flow rate of particles (i.e., the number of particles to inject per timestep) should be determined such that the total number of particles within the domain at any given time does not

make the solution prohibitively expensive. Note that this might result in injecting less than 1 particle per timestep if your timestep is very small. Particle trajectories should be integrated using a Runge-Kutta method of at least second order accuracy.

In summary: At each time step, you will solve the fluid velocity as described in §1. The number of particles to be injected into the domain will then be determined, and a new particle array will be created based on the current number of particles. The fluid velocity will then be interpolated to the location of each particle, and each particle will be advanced in time based on Eq. (40). The total number of particles that left the domain will then be determined, and a new particle array will be created based on the current number of particles.

Run your simulation until steady-state is reached with $\mathrm{Re} = 10$, $\mathrm{Re} = 1000$, and $\mathrm{Re} = \infty$ by varying the kinematic viscosity. Here, the Reynolds number is based on the centerline fluid velocity $U = 1$ and R, and thus $\mathrm{Re} = UR/\nu$. For each Reynolds number, vary the Stokes number between $0.1 < St < 100$.

Please report on the following aspects:

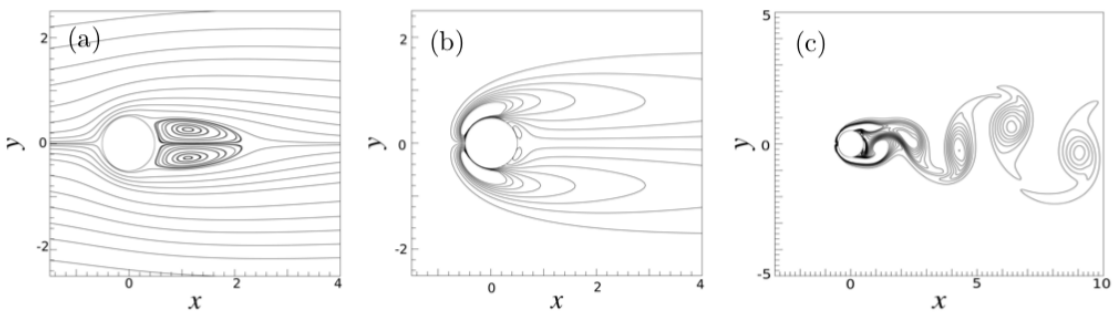
(a) Provide a 2D plot of the fluid velocity field and particles at different values of tU/R.

(b) How does the flow compare at different Reynolds numbers? Comment on this behavior. Is this expected?

(c) How does the flow compare at different Stokes numbers? Comment on this behavior. Is this expected?

2.4 Immersed boundary method with direct forcing

Simulations of fluid flow in complex geometries arise in many applications. In this example, you will solve the 2D Navier–Stokes equations using an immersed boundary (IB) method to account for the flow past an arbitrary shape. A number of IB methods exist, in this example I propose you follow the *direct forcing* approach originally proposed by Uhlmann in 2005: M. Uhlmann "An immersed boundary method with direct forcing for the simulation of particulate flows." *Journal of Computational Physics* 209.2 (2005): 448-476. A flow configuration corresponding to a wake behind a cylinder will be considered at various Reynolds numbers (see figure below). Solve the equations using second order accuracy in time (AB for convection, CN for diffusion using ADI), second order central differencing in space for the diffusion term and convective terms, as described in Sec. 1.



The main idea is to add an additional IB forcing term, $\boldsymbol{f}$, on the right-hand side of Navier–Stokes to model no-slip and no-penetration:

$$\frac{\partial \boldsymbol{u}}{\partial t} + \nabla \cdot (\boldsymbol{u} \otimes \boldsymbol{u}) = -\nabla p + \nu \nabla^2 \boldsymbol{u} + \boldsymbol{f}. \tag{43}$$

Using the fraction-step method described in Sec. 1.3, prior to the pressure correction (just after Eq. 13), Eq. 43 can be expressed as

$$\frac{\boldsymbol{u}^{\star\star}_{i,j} - \boldsymbol{u}^{\star}_{i,j}}{\Delta t} = \boldsymbol{f}_{ij}, \tag{44}$$

where $\boldsymbol{u}^\star$ is the velocity field after accounting for convection and viscosity, and $\boldsymbol{u}^{\star\star}$ is the velocity that accounts for the IB surface. Recognizing that we want the updated velocity to satisfy the no-slip condition (i.e., $\boldsymbol{u}^{\star\star} = \boldsymbol{u}_{IB}$), it is easy to show

$$\boldsymbol{F} = \frac{\boldsymbol{u}_{IB} - \boldsymbol{U}}{\Delta t}, \tag{45}$$

where $\boldsymbol{F}$ is the forcing at the IB surface, and $\boldsymbol{U}$ is the intermediate velocity $\boldsymbol{u}^\star$ located at the IB surface. Note that for a stationary object $\boldsymbol{u}_{IB} = 0$. The challenge is that $\boldsymbol{F}$ is not aligned with the computational grid. The direct forcing approach approximates the IB surface by placing marker particles at select locations. Each particle has a position $\boldsymbol{X}_p$ and volume V_p. To this end, $\boldsymbol{u}^\star$ needs to be interpolated to the particle location, and $\boldsymbol{F}$ needs to be extrapolated from the particle to the mesh. This is commonly done using the following discretization:

$$\boldsymbol{U} = \sum_{i_p-1}^{i_p+1} \sum_{j_p-1}^{j_p+1} \boldsymbol{u}^\star(\boldsymbol{x}) \delta^{2D}\left(\boldsymbol{x} - \boldsymbol{X}_p^{(i)}\right) V_c \quad \text{and} \quad \boldsymbol{f} = \sum_{i=1}^{N_p} \boldsymbol{F}(\boldsymbol{X}_p^{(i)}) \delta^{2D}\left(\boldsymbol{x} - \boldsymbol{X}_p^{(i)}\right) V_p^{(i)}, \tag{46}$$

where (i_p, j_p) is the grid cell each particle belongs to and $V_c = \Delta x \Delta y$ is the cell volume. Because we are considering a uniform mesh, (i_p, j_p) can easily be computed using the particle location and grid spacing, i.e., $i_p = \text{ceil}(X_p/\Delta x)$ and $j_p = \text{ceil}(Y_p/\Delta y)$, where $\text{ceil}(x)$ is the ceiling operator that rounds each element of x to the nearest integer greater than or equal to that element. In the above expression, δ^{2D} is a weighting function that represents a mollified (smooth) Delta function. This is typically computed using a so-called 'Roma' kernel, given by:

$$\delta^{2D}\left(\boldsymbol{x} - \boldsymbol{X}_p\right) = \delta^{1D}(x - X_p)\delta^{1D}(y - Y_p), \tag{47}$$

where the one-dimensional delta function is

$$\delta^{1D}(r) = \begin{cases} \frac{1}{3\Delta x}(1 + \sqrt{-3(r/\Delta x)^2 + 1}), & |r| \leq 0.5\Delta x \\ \frac{1}{6\Delta x}(5 - 3|r/\Delta x| - \sqrt{-3(1 - |r/\Delta x|)^2 + 1}), & 0.5\Delta x \leq |r| \leq 1.5\Delta x \\ 0, & \text{otherwise.} \end{cases} \tag{48}$$

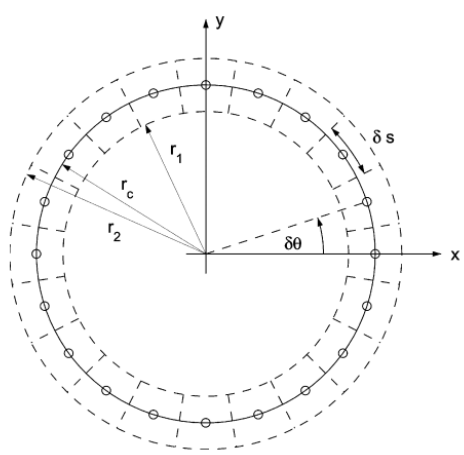
For our particular problem, consider a domain of $10D \times 10D$ centered at the cylinder of radius $R = D/2$, similar to what was shown above. Three Reynolds numbers should be considered: $\text{Re} = 5$, 30, and 200, with $\text{Re} = UD/\nu$. For adequate resolution, use 128 grid points in each direction, such that the cylinder is resolved with approximately 12.8 grid points. The marker particles will be distributed evenly along the circumference of the cylinder as depicted below. The arc-length, measured at radius R, is given by $\delta s = 2\pi/N_p$, where N_p is the total number of particles. Thus, the position of particle i is given by $X_p^{(i)} = R\cos\theta_i$ and $Y_p^{(i)} = R\sin\theta_i$, where θ_i is uniformly spaced between 0 and 2π. With these definitions, the surface of an element V_p for each particle is:

$$V_p = \frac{2\pi R \Delta x}{N_p} \tag{49}$$

For proper resolution there should be at least 1 particle per cell, which results in the following criterion for the total number of particles:

$$N_p \approx \frac{2\pi R}{\Delta x} \tag{50}$$

Finally, note that it is not necessary to fill the entire circle with marker particles, instead they



are only placed at the surface. This will result in finite velocity within the solid object, which is unphysical but should not contaminate the flow outside the cylinder.

Please report on the following aspects:

(a) Provide a 2D plot of the velocity and pressure fields at different values of tU/D for $Re = 10$, 30, and 200. How long does it take to reach steady state for each case? Explain how you determined steady state.

(b) How does the flow compare at different Reynolds numbers? Is this expected? How high of a Reynolds number can you achieve?

(c) What happens as you increase/decrease the total number of particles N_p? Is there a benefit to increasing N_p above the value given in Eq. 50?

2.5 Implementation using GPUs

GPU-accelerated computing is the use of a graphics processing unit (GPU) together with a CPU to accelerate performance of a code (see `http://www.nvidia.com/object/what-is-gpu-computing.html` for details on GPU basics). In today's world of high-performance computing, more and more supercomputers coming online are built with GPUs. For example, Oak Ridge National Lab's high performance supercomputer Summit is the world's largest computer to date. Each node on Summit will contain NVIDIA Volta GPUs. Therefore, if one wants to be at the cutting edge of scientific computing, they should have the tools capable of leveraging these resources.

For this example, you will implement the 2D Navier–Stokes equations described in §1 for any of the flow configurations given in the above examples, using GPUs. In general, writing a code for GPUs requires interfacing C or Fortran with the CUDA library or OpenACC. To date, these libraries require NVIDIA graphics cards. In most cases, the bottleneck in speed is associated with communication between the CPU and GPU. To keep things simple (and fast), the entire flow solver should be implemented on a single GPU using either OpenACC or CUDA.